Protocols, cryptanalysis and mathematical cryptology (INFO-F514)

Zero-Knowledge Proofs

Christophe Petit

Université libre de Bruxelles

Sensitization campaign

Together against sexual harassment and violences
https://www.ulb.be/fr/sante-et-bien-etre/si-cest-pas-oui-cest-non

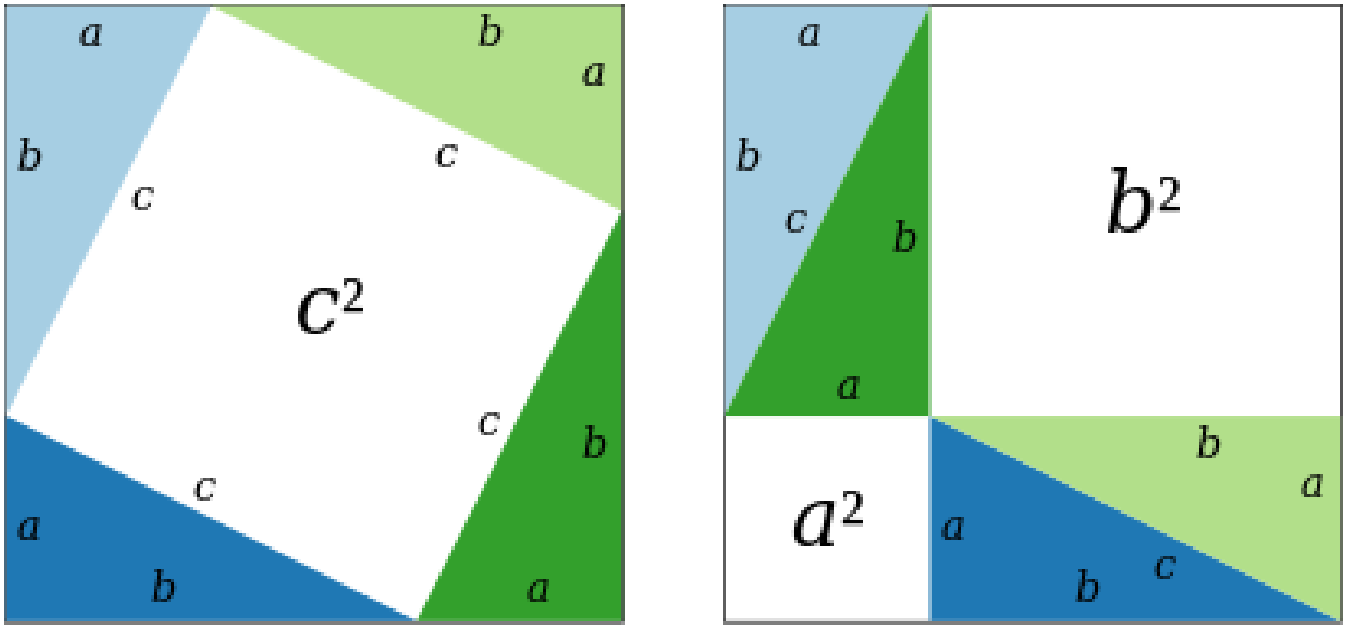
Protocols, cryptanalysis and mathematical cryptology (INFO-F514)

Zero-Knowledge Proofs

Christophe Petit

Université libre de Bruxelles

“Proof”? (1)



$$c^2 = a^2 + b^2$$

Picture credit: Wikipedia

"Proof"? (2)

- Statement:
  I know a secret value x such that $g^x = h \bmod p$
- A valid proof:
  Here is x. Now you can check that it satisfies the statement.
- Issue? x is no longer secret

"Proof"? (3)

- Statement:
  $(g_1, g_2, g_3, g_4) \in G^4$ *is a Diffie-Hellman tuple*
  i.e. $\exists x$ *such that* $g_2 = g_1^x$ *and* $g_4 = g_3^x$.

- A valid proof:
  Here is x. *Now you can check that it satisfies the statement.*

- Issue? revealing x when you only need to prove it exists (in e-voting contexts x could reveal a vote or secret key)

“Proof”? (4)

- Some valid proofs of identity and address in the UK:

Proof of identity checklist for individuals

You cannot use one form of identification for both name and address.

For example, if you provide your driving licence as proof of your name you must provide another form of identification for your address, such as a utility bill.

Proof of name	Proof of address
Current signed passport	Utility bill (gas, electric, satellite television, landline phone bill) issued within the last three months
Original birth certificate (UK birth certificate issued within 12 months of the date of birth in full form including those issued by UK authorities overseas such as Embassies High Commissions and HM Forces)	Local authority council tax bill for the current council tax year
EEA member state identity card (which can also be used as evidence of address if it carries this)	Current UK driving licence (but only if not used for the name evidence)
Current UK or EEA photocard driving licence	Bank, Building Society or Credit Union statement or passbook dated within the last three months
Full old-style driving licence	Original mortgage statement from a recognised lender issued for the last full year

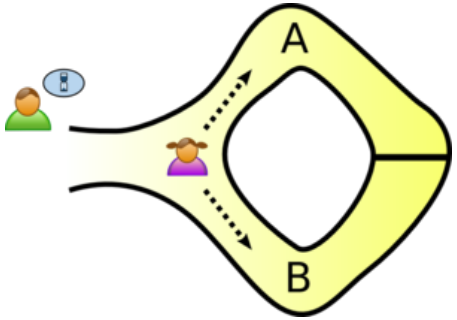
Source: `www.gov.uk/government/publications/proof-of-identity-checklist`

- A lot of extra information is revealed!

Zero-knowledge proof

- Goal: prove some statement, but without revealing any extra information to the verifier
- Examples
  - Prove that you know some password, but without revealing it (nor in fact anything about it)
  - Prove that you belong to an authorized set of users (e.g. over 18)
  - Prove that you have constructed a valid encrypted vote without revealing the vote
  - Signatures: implicit proof that you know the secret key associated to the public key

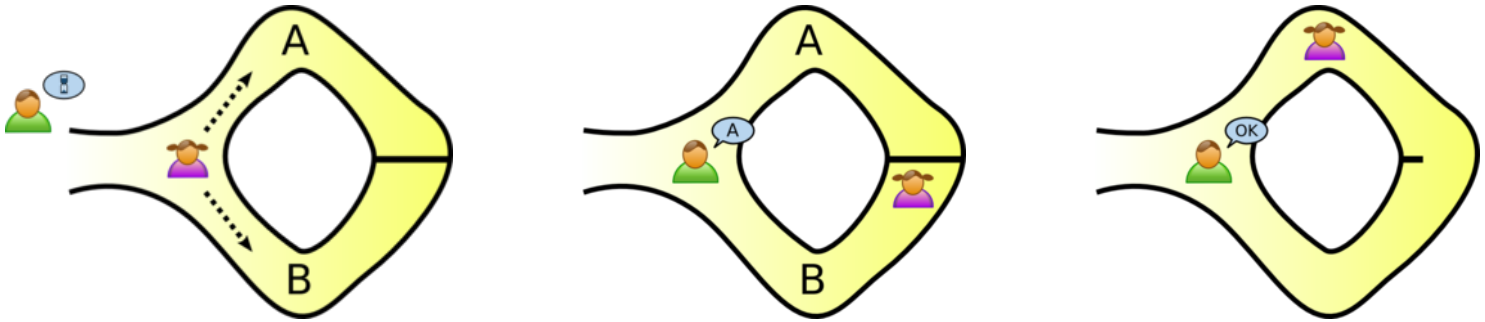
A first example: Mic Ali's cave



- Consider a cave as above, with two entries and a wall in the middle, which can be opened by a secret password
- Peggy (the prover) wants to prove to Victor (the verifier) that she knows the password, but without revealing it
- + she does not want Victor to further prove to others that she knows the password

Picture credit: Wikipedia. Example from Quisquater-Guillou "How to teach ZK to your children"

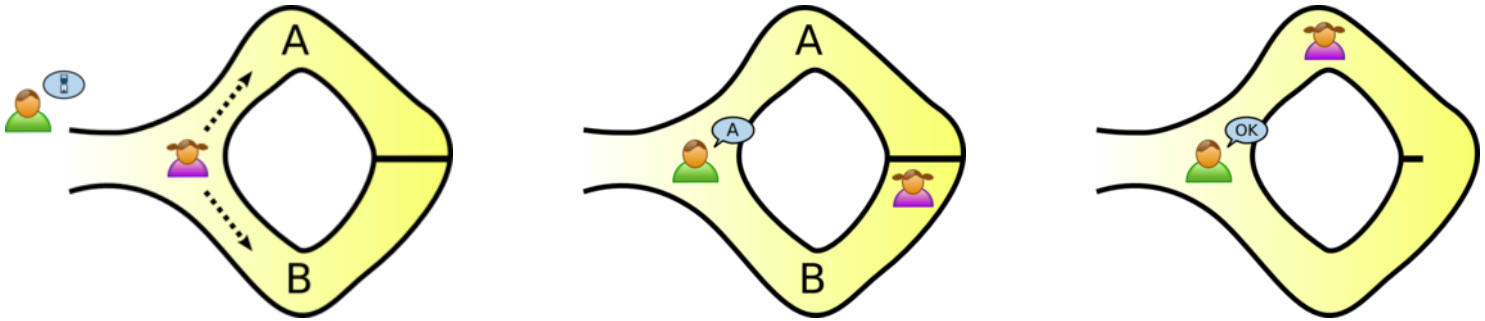
A first example: Mic Ali's cave (2)



- Proof:
  - Peggy privately enters one entry at random
  - Victor chooses one entry at random and tells (shouts) Peggy to exit that way
  - Peggy comes by requested exit, using password if needed
- Without password, Peggy cheats with 50% chance
- Repeating k times, the probability to pass all tests is 2^{-k}

Pictures credit: Wikipedia. Example from Quisquater-Guillou "How to teach ZK to your childen".

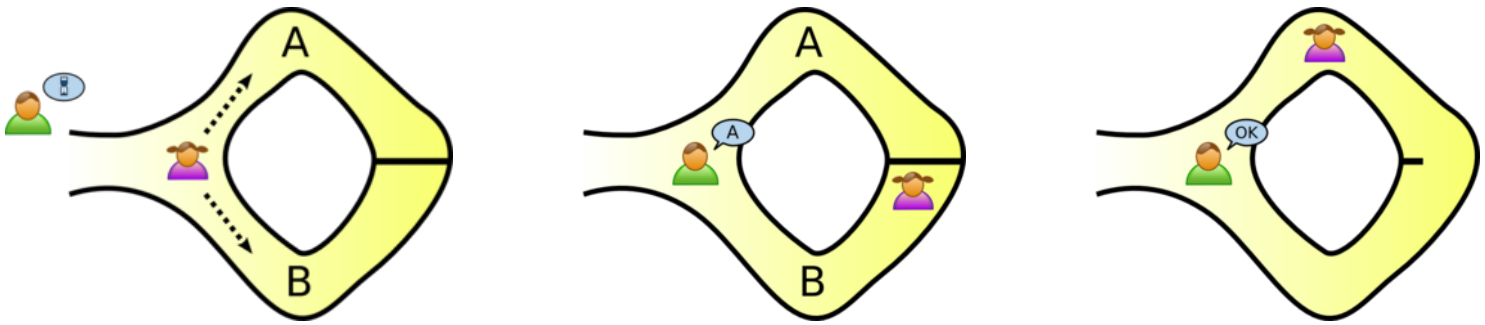
Correctness and soundness



- Protocol is *correct*: if Peggy knows the password she will be able to convince Victor that she knows it
- Protocol is *sound*: if Peggy does not know the password she will not be able to convince Victor that she knows it (except with a negligible probability)

Pictures credit: Wikipedia. Example from Quisquater-Guillou "How to teach ZK to your childen".

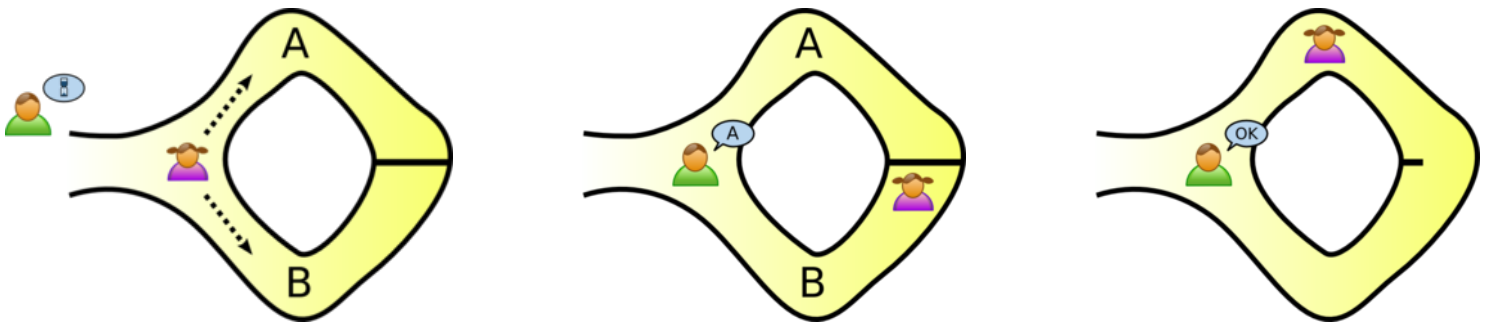
Zero-knowledge



- Observing the **interaction** between Peggy and Victor **does not reveal *anything*** about the password
- Indeed, this **interaction can be simulated *without the password***: choose the exit direction beforehand at random, and let Peggy enter this way

Pictures credit: Wikipedia. Example from Quisquater-Guillou "How to teach ZK to your childen".

Proof is non-transferable



- Victor cannot use the proof to convince someone else that Peggy knows the password
  - Even if they witness the whole process, they have no way to detect a potential collusion between Peggy and Victor
- Alternative solution to let Peggy go round the loop will convince Victor, but it will also convince other witnesses

Pictures credit: Wikipedia. Example from Quisquater-Guillou "How to teach ZK to your children".

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring

ZK protocols for conjonctions (AND) and disjunctions (OR)

ZK protocols for DLP-related relations

The Fiat-Shamir transform

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring

ZK protocols for conjonctions (AND) and disjunctions (OR)

ZK protocols for DLP-related relations

The Fiat-Shamir transform

Statements

- Statements to prove will be modeled in the form $x \in \mathcal{L}$ where

$$\mathcal{L} = \{x \in \mathcal{X} \mid \exists w \in \mathcal{W} \text{ s.t. } R(x, w) = 1\}$$

  Here
  - $R : \mathcal{X} \times \mathcal{W} \to \{0, 1\}$ is a **binary relation**
  - w is called a **witness** for $x \in \mathcal{L}$

- Example: $\mathcal{L}$ is a set of caves with a password wall
- Example (DH tuples): take $\mathcal{X} = (\mathbb{F}_p^*)^4$, $\mathcal{W} = \mathbb{Z}_{p-1}$ and $R((g_1, h_1, g_2, h_2), w) = 1 \Leftrightarrow h_1 = g_1^w$ and $h_2 = g_2^w$

Zero-Knowledge Proofs

- Prover knows a pair (x, w) such that $R(x, w) = 1$; Verifier only gets x
- Prover wants to prove $x \in \mathcal{L}$ without revealing any information about the witness w
- Proofs are often interactive: several messages between prover and verifier (and the order matters!)

Remarks

- We typically consider only NP statements
  (R can be evaluated in polynomial time)
- Typically hard for verifier to compute w given x
  (otherwise the Prover is not needed to verify that $x \in \mathcal{L}$)

"Sigma protocol"

- Goal: prover wants to convince verifier that $x \in \mathcal{L}$, knowing w such that $R(x, w) = 1$
- 3-round communication protocol
  1. Prover sends a **commitment** to verifier
     (think of Peggy entering the cave on one side)
  2. Verifier sends a **challenge** to prover
     (think of Victor asking her to exit on one side)
  3. Prover sends a **response** to verifier
     (think of Peggy exiting on the correct side)
- At the end Verifier either accepts or rejects the proof

Security properties

- **Correctness:** if the prover knows the witness and follows the protocol then proof will be accepted by the verifier
- **Soundness:** if the prover does not know the witness then the verifier will not accept the proof
- **Zero-knowledge:** nothing is revealed about witness w (except that it exists)

Identification protocol

- Key generation algorithm produces a pair of public, private keys (x, w)
- Interactive protocol aims to prove knowledge of secret key w corresponding to public key x
- Security property: cannot pass the verification without knowing the secret key, even after observing other protocol instances running
- Solution: run a ZK protocol for the specific relation

$$R(x, w) = 1 \Leftrightarrow w \text{ is a private key for public key } x$$

Formally: correctness

- Prover $\mathcal{P}$ and Verifier $\mathcal{V}$ are probabilistic polynomial time algorithms
- Correctness:

$$\Pr\left[\mathcal{V}(x,\alpha,\beta,\gamma)=1 \;\middle|\; \begin{array}{l} \alpha \leftarrow \mathcal{P}(x,w); \\ \beta \leftarrow \mathcal{V}(x,\alpha); \\ \gamma \leftarrow \mathcal{P}(x,w,\alpha,\beta) \end{array}\right] = 1$$

k-special soundness

- Soundness: if the prover does not know the witness then the verifier will not accept the proof
- k-***special*** **soundness:** one can efficiently *extract* a valid witness from k valid responses to k distinct challenges (all of them for the same initial commitment)
  - If a witness can be extracted then $x \in \mathcal{L}$
  - If Prover successful on random challenge, we can expect it is also able to produce valid answers to k of them
  - Prover could simulate verifier and produce k responses, then extract the witness (hence: Prover knows witness)
- Note that k challenges and responses are NOT exchanged in actual protocol taking place (this would leak witness!)
- The Ali Baba protocol is sound but not special sound

Honest Verifier Zero-Knowledge (HVZK)

- Zero-knowledge: nothing is revealed about witness *w* (except that it exists)
- **Honest Verifier Zero-Knowledge:** there exists an efficient *simulator* algorithm which can produce a conversation (α, β, γ) *indistinguishable* from an actual prover-verifier conversation *without knowing witness w*
  - If witness is not used to produce the triple (α, β, γ) then no information about it is leaked
- Note that the simulator can produce α, β, γ in any order, so no contradiction with soundness
- The Ali Baba protocol is HVZK

Formally

- k-special soundness: there exists a probabilistic polynomial time algorithm $\mathcal{E}$ (the extractor) such that ...(exercice!)
- Honest-verifier zero-knowledge there exists a probabilistic polynomial time algorithm $\mathcal{S}$ (the simulator) such that ...(exercice!)

Perfect/statistical/computational properties

- Depending on the protocols, the properties above may hold perfectly, statistically or computationally
  - Perfectly: property always true
  - Statistically: true with overwhelming probability
  - Computationally: true, unless some computational problem is solved (like computing discrete logarithms)
- Impossibility result: cannot have perfect soundness and perfect zero-knowledge together
- A ZK proof that only has computational soundness is sometimes called a ZK *argument* instead

Remark

- Literature includes other definitions of correctness, soundness and zero-knowledge, with (sometimes subtle) differences, but an intent to capture the same intuitions

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring

ZK protocols for conjonctions (AND) and disjunctions (OR)

ZK protocols for DLP-related relations

The Fiat-Shamir transform

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring
- Graph isomorphism
- 3-coloring and arbitrary NP statements

ZK protocols for conjonctions (AND) and disjunctions (OR)

ZK protocols for DLP-related relations

The Fiat-Shamir transform

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring
- Graph isomorphism
- 3-coloring and arbitrary NP statements

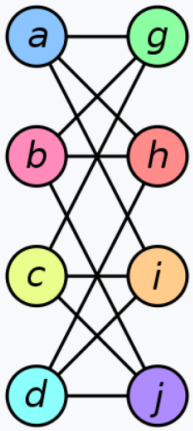
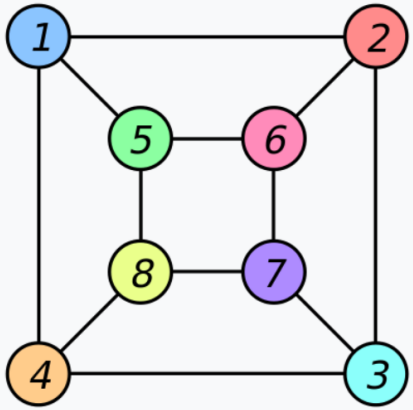
ZK protocols for conjonctions (AND) and disjunctions (OR)

ZK protocols for DLP-related relations

The Fiat-Shamir transform

Graph isomorphism

- Graph isomorphism: function sending vertices of one graph to another graph and preserving edge relations

Graph G	Graph H	An isomorphism between G and H
		$f(a) = 1$ $f(b) = 6$ $f(c) = 8$ $f(d) = 3$ $f(g) = 5$ $f(h) = 2$ $f(i) = 4$ $f(j) = 7$

Picture credit: Wikipedia

Graph isomorphism problem

- One of the most famous problems in complexity theory: decide when two graphs are isomorphic to each other
- Not known to be in P, not believed to be NP-complete
- Breathrough quasi-polynomial algorithm by Babai (2015-2017)

- Let's pretend this algorithm does not exist and that we have a solution to a graph isomorphism problem. How to prove this without revealing the solution?

Zero-knowledge proof for graph isomorphism

- Let π be an isomorphism from G_0 to G_1
  (π is a witness that G_0 and G_1 are isomorphic)
- Prover constructs a random isomorphism $\sigma_1 : G_1 \to H$
  (concretely, a random permutation of the vertices)
- Prover commits with graph H
- Verifier sends a challenge bit b
- If $b = 0$ then Prover computes $\sigma_0 = \sigma_1 \circ \pi$
- Prover responds with isomorphism $\sigma_b : G_b \to H$
- Repeat the protocol k times

Security properties

- Correctness: clear
- 2-special soundness: given valid answers to challenges $b = 0$ and $b = 1$ one can compute $\pi = \sigma_1^{-1} \circ \sigma_0$
- HVZK: a simulator can choose a random b, then a random σ_b, then deduce H and return (H, b, σ_b)

- Note: simulator does not compute messages in the same order as in a normal execution of the protocol, but message triple distribution is still as expected
- Prover can cheat with probability 50% by guessing b if $k = 1$; this is reduced to 2^{-k} after k repetitions

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring
- Graph isomorphism
- 3-coloring and arbitrary NP statements

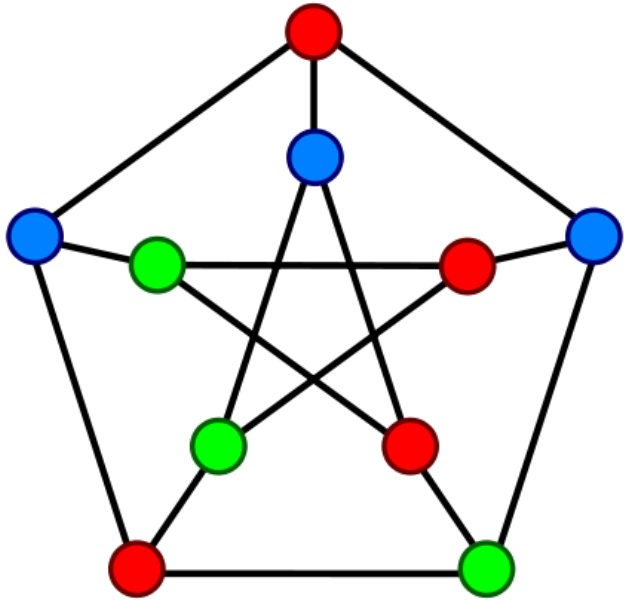
ZK protocols for conjonctions (AND) and disjunctions (OR)

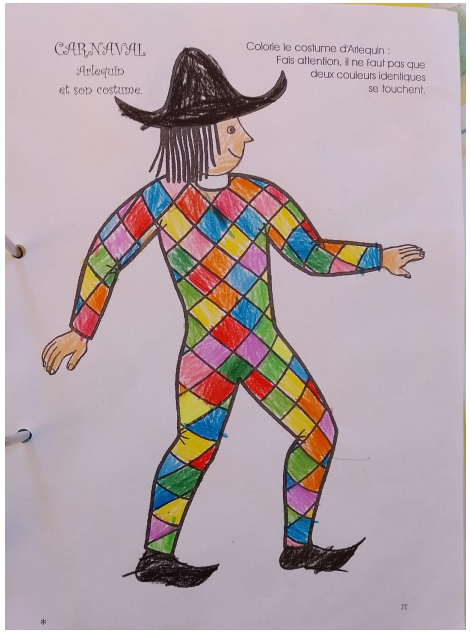
ZK protocols for DLP-related relations

The Fiat-Shamir transform

3-coloring

- k-coloring: color vertices of a graph with k colors such that no two adjacent vertices have the same color (or determine that no such coloring exists)





- For $k = 3$, typical example of an NP-complete problem

Pictures credit: Wikipedia; Nino Petit (4y old) and Mesdames Erika and Monique

ZK Proof for 3-coloring

- Let $G = (V, E)$ be a graph, and let

$$\phi : V \to \{0, 1, 2\}$$

  be a valid 3-coloring of G

- Let f be a one-way permutation (such as an encryption function with decryption key unknown to the verifier)

ZK Proof for 3-coloring (2)

- Prover chooses random $\pi : \{0,1,2\} \to \{0,1,2\}$ and lets $\phi' = \pi \circ \phi$
- Prover sends *commitments* $\{f(\phi'(v)||r_v) \mid v \in V\}$ where r_v are sufficiently large random bitstrings (note commitments are both *hiding* and *biding*)
- Verifier challenges on one edge $e \in E$
- Prover reveals $\phi'(v)||r_v$ for both v on the edge e
- Verifier checks the two vertices have distinct colors, Verifier also computes $f(\phi'(v)||r_v)$ and compares this with the committed values
- Prover and verifier repeat this protocol $|E|^2$ times

Security properties

- Correctness: clear
- Special soundness: valid responses to all edges will lead to a valid 3-coloring $\phi' = \pi \circ \phi$
- HVZK: simulator picks random e, gives two distinct random colours to the associated vertices, chooses random r_v, computes the commitments

Remarks

- Simulator does not compute messages in the same order as in a normal execution of the protocol, but message triple distribution is still as expected
- HVZK is only computational: a powerful adversary could invert the one-way permutation

All NP statements have zero-knowledge proofs

Proofs that Yield Nothing But Their Validity or All Languages in NP Have Zero-Knowledge Proof Systems

ODED GOLDREICH

Technion, Haifa, Israel

SILVIO MICALI

Massachusetts Institute of Technology, Cambridge, Massachusetts

AND

AVI WIGDERSON

Hebrew University, Jerusalem, Israel

Abstract. In this paper the generality and wide applicability of *zero-knowledge proofs*, a notion introduced by Goldwasser, Micali, and Rackoff is demonstrated. These are probabilistic and interactive proofs that, for the members of a language, efficiently demonstrate membership in the language without conveying any additional knowledge. All previously known zero-knowledge proofs were only for number-theoretic languages in NP $\cap$ CoNP.

Under the assumption that secure encryption functions exist or by using ''physical means for hiding information,'' it is shown that all languages in NP have zero-knowledge proofs. Loosely speaking, it is possible to demonstrate that a CNF formula is satisfiable without revealing any other property of the formula, in particular, without yielding neither a satisfying assignment nor properties such as whether there is a satisfying assignment in which $x_1 = x_3$ etc.

It is also demonstrated that zero-knowledge proofs exist ''outside the domain of cryptography and number theory.'' Using no assumptions, it is shown that both graph isomorphism and graph nonisomorphism have zero-knowledge interactive proofs. The mere existence of an interactive proof for graph nonisomorphism is interesting, since graph nonisomorphism is not known to be in NP and hence no efficient proofs were known before for demonstrating that two graphs are not isomorphic.

ZK proofs for NP statements

- Any NP statement can be reduced to a 3-coloring problem (since 3-coloring is NP complete)
- Both Prover and Verifier can do this reduction
- Prover then proves 3-coloring with ZK protocol

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring

ZK protocols for conjonctions (AND) and disjunctions (OR)

ZK protocols for DLP-related relations

The Fiat-Shamir transform

AND proofs ("conjonctions")

- Let $R_0 : \mathcal{X}_0 \times \mathcal{W}_0 \to \{0,1\}$ and $R_1 : \mathcal{X}_1 \times \mathcal{W}_1 \to \{0,1\}$ be two relations
- Let as before

$$\mathcal{L}_i = \{x \in \mathcal{X}_i \mid \exists w \in \mathcal{W}_i \text{ s.t. } R_i(x,w) = 1\}$$

- Let

$$\mathcal{L} = \{(x_0, x_1) \in \mathcal{X}_0 \times \mathcal{X}_1 \mid x_0 \in \mathcal{L}_0 \text{ AND } x_1 \in \mathcal{L}_1\}$$

- Goal: given ZK proof protocols for $\mathcal{L}_0$ and $\mathcal{L}_1$, construct a ZK proof protocol for $\mathcal{L}$

Conjonctions: concatenate both proofs

- Run both proofs sequentially
  - Correctness: clear
  - Soundness: clear
  - HVZK: clear
- Can usually run the two proofs in parallel, and can sometimes optimize further

AND proofs: parallel solution

Prover(x_0, x_1, w_0, w_1)		Verifier(x_0, x_1)
$\alpha_0 \leftarrow P_0(x_0, w_0)$; $\alpha_1 \leftarrow P_1(x_1, w_1)$;		
	$\xrightarrow{\alpha_0, \alpha_1}$	
		$\beta_0 \leftarrow V_0(x_0, \alpha_0)$; $\beta_1 \leftarrow V_1(x_1, \alpha_1)$;
	$\xleftarrow{\beta_0, \beta_1}$	
$\gamma_0 \leftarrow P_0(x_0, w_0, \alpha_0, \beta_0)$; $\gamma_1 \leftarrow P_1(x_1, w_1, \alpha_1, \beta_1)$;		
	$\xrightarrow{\gamma_0, \gamma_1}$	
		Accept if $V_0(\alpha_0, \beta_0, \gamma_0) = 0$ and $V_1(\alpha_1, \beta_1, \gamma_1) = 0$

OR proofs ("disjunctions")

- Let $R_0 : \mathcal{X}_0 \times \mathcal{W}_0 \to \{0,1\}$ and $R_1 : \mathcal{X}_1 \times \mathcal{W}_1 \to \{0,1\}$ be two relations
- Let as before

$$\mathcal{L}_i = \{x \in \mathcal{X}_i \mid \exists w \in \mathcal{W}_i \text{ s.t. } R_i(x,w) = 1\}$$

- Let

$$\mathcal{L} = \{(x_0, x_1) \in \mathcal{X}_0 \times \mathcal{X}_1 \mid x_0 \in \mathcal{L}_0 \text{ OR } x_1 \in \mathcal{L}_1\}$$

- Goal: given ZK proof protocols for $\mathcal{L}_0$ and $\mathcal{L}_1$, construct a ZK proof protocol for $\mathcal{L}$
- Remark: proof should not reveal which $x_i \in \mathcal{L}_i$ is true

OR proof: first attempt

- Prover knows either w_0 or w_1 but a priori not both
- Produce two proof transcripts
  - One by running the ZK proof protocol normally (on the statement for which a witness is known)
  - One by running the Zero-Knowledge simulator (on the statement for which no witness is known)
- Verifier can check both proof transcripts are valid

OR proof : first attempt

Prover(x_0, x_1, b, w_b)		Verifier(x_0, x_1)
$\alpha_b \leftarrow P_b(x_b, w_b)$;		
	$\xrightarrow{\alpha_b}$	
		Choose random β_b;
	$\xleftarrow{\beta_b}$	
$\gamma_b \leftarrow P_b(x_b, w_b, \alpha_b, \beta_b)$; Choose random β_{1-b}; $(\alpha_{1-b}, \beta_{1-b}, \gamma_{1-b}) \leftarrow S_{1-b}(x_{1-b}, \beta_{1-b})$;		
	$\xrightarrow{\gamma_b, \alpha_{1-b}, \beta_{1-b}, \gamma_{1-b}}$	
		Accept if $V_0(\alpha_0, \beta_0, \gamma_0) = 1$ and $V_1(\alpha_1, \beta_1, \gamma_1) = 1$

Intuition (2)

- Issue with previous attempt: Verifier learns b!
- To avoid that, proof should look identical in both cases

- Send α_0 and α_1 together in first round
- Send γ_0 and γ_1 together in third round

- Use two challenges β_0 and β_1 such that $\beta_0 \oplus \beta_1 = \beta$, where β is sent by verifier
  - Prover can choose β_0 or β_1 for simulation, but not both
  - Verifier does not learn which challenge was chosen
  - Verifier can check $\beta_0 \oplus \beta_1 = \beta$

OR proof

Prover(x_0, x_1, b, w_b)		Verifier(x_0, x_1)
$\alpha_b \leftarrow P_b(x_b, w_b)$; Choose random β_{1-b}; $(\alpha_{1-b}, \beta_{1-b}, \gamma_{1-b}) \leftarrow S_{1-b}(x_{1-b}, \beta_{1-b})$;		
	$\xrightarrow{\alpha_0, \alpha_1}$	
		Choose random β;
	$\xleftarrow{\beta}$	
$\beta_b = \beta \oplus \beta_{1-b}$; $\gamma_b \leftarrow P_b(x_b, w_b, \alpha_b, \beta_b)$;		
	$\xrightarrow{\beta_0, \beta_1, \gamma_0, \gamma_1}$	
		Accept if $\beta_0 \oplus \beta_1 = \beta$ and $V_0(\alpha_0, \beta_0, \gamma_0) = 1$ and $V_1(\alpha_1, \beta_1, \gamma_1) = 1$

Security properties

- Correctness: clear
- Special soundness:
  - Suppose you have two valid transcripts $(\alpha_0, \alpha_1, \beta, \beta_0, \beta_1, \gamma_1, \gamma_2)$ and $(\alpha_0, \alpha_1, \beta', \beta'_0, \beta'_1, \gamma'_1, \gamma'_2)$ with $\beta \neq \beta'$
  - Then for at least one value $b \in \{0, 1\}$ we have $\beta_b \neq \beta'_b$
  - Run the special soundness extractor on $(\alpha_b, \beta_b, \gamma_b, \beta'_b, \gamma'_b)$
- HVZK: to simulate conversation, choose random β_0, β_1, run simulators on both proofs separately, set $\beta = \beta_0 \oplus \beta_1$

Generalization: "one-out-of-n" proofs

- Use $n-1$ simulated proofs and one actual proof
- Complexity linear in n

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring

ZK protocols for conjonctions (AND) and disjunctions (OR)

ZK protocols for DLP-related relations

The Fiat-Shamir transform

Motivation

- We already have a ZK proof protocol for arbitrary NP statements; why bother with special cases?
- Efficiency! General constructions give protocols with poor efficiency in terms of
  - Prover computation
  - Verifier computation
  - Communication costs
- Special structure in specific statements can lead to much better protocols
  - Example: group structure in DLP-based protocols

Proofs covered here

- Schnorr identification protocol (discrete log solution)
- Chaum-Pedersen protocol (Diffie-Hellman tuples)
- Correct decryption of an ElGamal ciphertext

Schnorr identification protocol

- Let G be a finite group of prime order q, and let g be a generator for that group
- Prover has a private key x, and a public key $h = g^x$
- Goal: prove knowledge of x without revealing anything about it

Schnorr identification protocol

- Prover chooses random r and computes $R = g^r$
- Prover sends R to verifier
- Verifier sends random challenge c to Prover
- Prover answers with $s = r + xc \bmod q$
- Verifier checks that $g^s = Rh^c$

Intuitions

- Revealing x would violate zero-knowledge
- So reveal a "masked version" s of x related to
  - An element precommitted to by the prover
  - A challenge sent by the verifier
- Element s is uniformly distributed

Example: ZK proof of discrete-logarithm in $\mathbb{Z}_{11}^\star$

- Let $G = \{1, 3, 4, 5, 9\} \subset \mathbb{Z}_{11}^\star$ of order 5

i	0	1	2	3	4	5	6	7	8	9	10
$5^i \mod 11$	1	5	3	4	9	1	5	3	4	9	1

- Let $g = 5$ a generator
- Let $x = 4$ so $h = g^x = 5^4 = 9 \mod 11$
- Prover chooses random $r = 3$ and sends commitment $R = g^r = 5^3 = 4 \mod 11$
- Verifier sends random challenge $c = 2$
- Prover sends $s = r + cx = 3 + 2 \cdot 4 = 1 \mod 5$
- Verifier checks $g^s = 5 = Rh^c = 4 \cdot 9^2 \mod 11$

Security properties

- Correctness: we have

$$g^s = g^{r+cx} = g^r(g^x)^c = Rh^c$$

- Special soundness: given two valid transcripts (R, c, s) and (R, c', s') with $c \neq c'$ can compute

$$\frac{s' - s}{c' - c} = \frac{(r + c'x) - (r + cx)}{c' - c} = x$$

(*computational* soundness: relies on DLP hardness)

- HVZK: simulator chooses s and challenge c, then computes R

Other useful properties

- Pairs (r, R) can be precomputed
- c sometimes restricted to be a small number
  - $+$ Efficiency advantage
  - $-$ s not uniform, so extra assumption needed for security
- Both properties useful when prover has small computing power (such as an RFID tag)

Chaum-Pedersen protocol

- Let G_1, G_2 be two groups of prime order q (we could have $G_1 = G_2$, but not necessarily)
- Let $g_1, h_1 \in G_1$ and $g_2, h_2 \in G_2$ where $h_1 = g_1^x$ and $h_2 = g_2^x$ for some x
- Goal: prove (g_1, h_1, g_2, h_2) are of this format, without revealing anything about x
- Application: DDH tuples (g, g^a, g^b, g^{ab})

Chaum-Pedersen protocol

- Idea: run two instances of Schnorr protocol in parallel, with the same challenge c and the same answer s (therefore proving that the same x is involved in both)

Chaum-Pedersen protocol

- Prover knows x such that $h_1 = g_1^x$ and $h_2 = g_2^x$
- Prover chooses random r
- Prover sends $R_1 = g_1^r$ and $R_2 = g_2^r$ to verifier
- Verifier sends a random challenge c
- Prover answers with $s = r + cx$
- Verifier checks $g_1^s = R_1 h_1^c$ and $g_2^s = R_2 h_2^c$

Security properties

- Correctness: as for Schnorr
- Special soundness: following special soundness proof for Schnorr we can extract x_1, x_2 and

$$x_1 = x_2 = \frac{s' - s}{c' - c}$$

- HVZK: as for Schnorr

Reminder: ElGamal encryption

- Key generation:
  - Generate cyclic group G of prime order q
  - Generate random $x \in [1, q]$
  - Pick a generator g and set $h = g^x$
  - Private key is x and public key is (G, g, h)
- Encryption: given public key (G, g, h) and message m
  - Pick random $r \in [1, q]$
  - Return $c = (c_1, c_2) = (mh^r, g^r)$
- Decryption: given private key x and ciphertext c
  - Return $m' = c_1 c_2^{-x}$

Proofs related to ElGamal

- Goal: prove that (c_1, c_2) is a valid encryption of m
  1. Knowing secret key x (but not randomness r)
  2. Knowing randomness r (but not secret key x)
- Solution: use Chaum-Pedersen proof with
  1. $g_1 = g$, $h_1 = h$, $g_2 = c_2$, $h_2 = c_1 m^{-1}$
  2. $g_1 = g$, $h_1 = c_2$, $g_2 = h$, $h_2 = c_1 m^{-1}$

Application: evoting

- Many e-voting schemes use a variant of ElGamal where m is replaced by g^m in ciphertexts (called multiplicative ElGamal, just as secure, though decryption only feasible when m is small)
- Every candidate is associated an ElGamal public key
- Voters encrypt $m = 0$ or $m = 1$ depending on whether they want to vote for the candidate
- ElGamal homomorphic: can combine individual ciphertexts to produce ciphertext of the sum
- Voters then need to prove that their ciphertext contains either 0 or 1 (and not e.g. 1000 or -1)
- Election authority decrypts total with ElGamal secret key, they also prove the result is correct

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring

ZK protocols for conjonctions (AND) and disjunctions (OR)

ZK protocols for DLP-related relations

The Fiat-Shamir transform

Remember: sigma protocols

- Goal: prover wants to convince verifier that $x \in \mathcal{L}$, knowing w such that $R(x, w) = 1$
- 3-round communication protocol
  - Prover sends a **commitment** to verifier
  - Verifier sends a **challenge** to prover
  - Prover sends a **response** to verifier
- At the end Verifier either accepts or rejects the proof

- Can we remove interaction?
  Can we let prover choose the challenge?

Fiat-Shamir heuristic

- Goal: prover wants to convince verifier that $x \in \mathcal{L}$, knowing w such that $R(x, w) = 1$
- 3-round communication protocol
  - Prover sends a **commitment** to verifier
  - Verifier sends a **challenge** to prover
  - Prover sends a **response** to verifier
- At the end Verifier either accepts or rejects the proof

- Let verifier *compute* the challenge, but not *choose* it: replace challenge by a hash of the transcript so far
- Send commitment and response as a single message

Fiat-Shamir heuristic

- Goal: prover wants to convince verifier that $x \in \mathcal{L}$, knowing w such that $R(x, w) = 1$
- You know a sigma protocol (P, V)
- You know a hash function H "random oracle"

- Prover computes $\alpha \leftarrow P(x, w)$
- Prover computes $\beta = H(x, \alpha)$
- Prover computes $\gamma \leftarrow P(x, w, \alpha, \beta)$
- Prover sends *non-interactive* proof $\pi = (\alpha, \gamma)$ to Verifier
- Verifier computes $\beta = H(x, \alpha)$, checks $V(x, \alpha, \beta, \gamma) = 1$

Security

- One can prove security of this construction under appropriate assumptions on H
- Intuitively: works when H is a random function, so challenge is still uniformly distributed and cannot be chosen by the prover (otherwise proof could be simulated)
- In practice SHA is not a random function; intuition still makes sense but no rigorous proof

Reminder: digital signatures

- Triple of algorithms ($KeyGen$, Sig, $Verif$)
- $KeyGen$ generates pairs (private key, public key)
- Sig takes message and private key, outputs signature
- $Verif$ takes message, signature, public key; outputs $0/1$
- *Existential unforgeability under chosen message attacks*: cannot forge signatures, even when you can request signatures on other messages of your choice

Fiat-Shamir signatures

- Consider an identification protocol $(KeyGen, P, V)$
  - *KeyGen* generates (private key, public key) pairs
  - Sigma protocol proving knowledge of the secret key associated to a public key
  - Use message as a source of randomness
- Derive a signature scheme as follows:
  - *KeyGen*: just run *KeyGen* of the identification protocol
  - *Sign*: return $\sigma = (\alpha, \gamma)$ where
    - α is the commitment in ID protocol
    - $\beta = H(m, PK, \alpha)$
    - γ is response to challenge β in ID protocol
  - *Verif*: recompute β and verify (α, β, γ)

Fiat-Shamir signatures: optimization

- If α can be recomputed from β and γ, signature (β, γ) instead of (α, γ)
- Fiat-Shamir signatures become
  - *KeyGen*: just run *KeyGen* of the identification protocol
  - *Sign*: return $\sigma = (\beta, \gamma)$ where
    - α is the commitment in ID protocol
    - $\beta = H(m, PK, \alpha)$
    - γ is response to challenge β in ID protocol
  - *Verif*: recompute α and check $\beta = H(m, PK, \alpha)$

Security of Fiat-Shamir signatures

- One can prove security of this construction under appropriate assumptions on H
- Intuitively: works with H a random function, so challenge still uniformly distributed and not chosen by the prover
- In practice SHA is not a random function;
  intuition still makes sense but no rigorous proof

Schnorr signatures

- Signatures resulting from the Fiat-Shamir heuristic (with optimization)
  applied on Schnorr's identification protocol

Reminder: Schnorr signatures

- Key generation algorithm:
  - Generate a cyclic group G of prime order q
  - Choose a generator g for G
  - Choose random secret key x
  - Compute $h = g^x$ and set public key to (g, h)
- Signature algorithm: $\mathrm{Sig}(m, x, (g, h))$:
  - Choose random r and compute $R = g^r$
  - Compute $c = H(m, g, h, R)$
  - Compute $s = r - xc$
  - Return $\sigma = (c, s)$
- Verification algorithm: $\mathrm{Verif}(m, \sigma, (g, h))$:
  - Compute $R' = g^s h^c$
  - Accept if and only if $c = H(m, g, h, R')$

Outline

Definitions

ZK protocols for graph isomorphism and 3-coloring

ZK protocols for conjonctions (AND) and disjunctions (OR)

ZK protocols for DLP-related relations

The Fiat-Shamir transform

Conclusion

- Zero-knowledge proofs reveal nothing but their validity
- Security: correctness, soundness, zero-knowledge
- Any NP statement has an efficient zero-knowledge proof
- Specific statements involving group elements have more efficient, dedicated proofs
- Fiat-Shamir heuristic transforms interactive proofs into non-interactive proofs, and identification protocols into signature schemes

References

- Goldreich-Micali-Widgerson, *Proofs that Yield Nothing But Their Validity: All Languages in NP Have Zero-Knowledge*
- Venturi, *Zero-Knowledge Proofs and Applications* (lecture notes)
- Damgaard, *On σ-protocols* (lecture notes)
- Cramer-Damgaard-Schoenmakers, *Proofs of partial knowledge and simplified design of witness hiding protocols*
- Fiat-Shamir, *How to prove yourself: practical solutions to identification and signature problems*
- Quisquater-Guillou-Annick-Berson, *How to explain zero-knowledge to your children*

Further study

- Zero-knowledge proofs are an active field of research; see main Crypto conference programs in recent years
- Research aims at
  - Improved efficiency (prover, verifier, communication) both for generic statements and dedicated applications
  - Better (or just other) hardness assumptions
- Ask me for references in this area!